

Repository Analysis and Verification Results

We examined the current codebase and test outputs. The system's state-reconstitution module loads a persisted ledger but **immediately trusts the “latest” row as authoritative**, without re-admission checks. For example, a failed `CreateVolume` call (HTTP 400) indicates the deployment code blindly retried provider mutations without validating the required API fields. In AWS EC2's `CreateVolume` API, **exactly one** of `AvailabilityZone` or `AvailabilityZoneId` must be present [\[40†L39-L43\]](#) ; sending neither (or both) yields a 400. The live tests (21 passing after recent fixes) show the *verification* steps (hash checks, byte consistency) work, but **no admission logic** examines higher-level predicates (authority, scope, resource existence). Thus the repository implements a *bounded byte-verification pipeline* but omits a typed admission gate.

We also confirmed that every test log is *explicitly inconclusive* about semantic conditions: e.g. when an assertion fails, no higher-level decision is made. The runtime prints that *structural input enforcement is not established* and *authority = NONE*, matching the expectation that the system *does not derive authorship or semantic truth from its traces* [\[6†L169-L174\]](#) [\[38†L139-L143\]](#) . In short, “restore → continue” has replaced the safer “restore → candidate → admit → authority → commit” pipeline.

Prior Art Survey: Admission and Gating

Heterogeneous component admission: Recent agent-governance work closely matches this architecture. The *HALO* framework (Heterogeneous Admission with Localized Obligations) treats each output component of an agent's response separately [\[38†L139-L143\]](#) [\[15†L785-L793\]](#) . For example, a UAV agent may output `[notice, report, action, status]` . If the support data for `action` changes, HALO blocks just that action (and its dependent `status`) while preserving `notice` and `report` [\[38†L128-L133\]](#) [\[38†L139-L143\]](#) . It then revalidates or requests a fresh candidate for the blocked action. In HALO's terms, a **component-level admission model** checks typed outputs against a trusted catalog and current state, removing invalid components and rechecking each exact action before dispatch [\[38†L139-L143\]](#) [\[15†L785-L793\]](#) . The figure below (from HALO) illustrates this process, where each response piece (e.g. flight action) is admitted or withheld as conditions change:

[\[16†embed_image\]](#) *Figure: Each agent output is a separate component. HALO's runtime preserves supported components and revalidates actions at dispatch* [\[38†L139-L143\]](#) [\[15†L785-L793\]](#) .

This “partial retention, exact dispatch, fresh readmission” approach [\[15†L785-L793\]](#) [\[38†L139-L143\]](#) provides strong prior art for *consequence gating*: it admits only the invariant consequence for a given observation class, and withholds changes until revalidation. Importantly, HALO also enforces **dependency coherence** (blocked components record a “localized obligation” for updates) and **one-dispatch tokens** for actions [\[15†L785-L793\]](#) .

Runtime uncertainty and logic: The need for three-valued logic under partial observability is well-established. Runtime monitors already produce verdicts {true, false, inconclusive} when trace data is

incomplete [6†L169-L174] . However, we emphasize that *our gating problem is consequence-relative***: the same partial observation may permit one consequence and forbid another. In epistemic terms, the system maintains a set of compatible worlds (observational equivalence class) and checks if all worlds agree on the consequence. If not, it withholds execution rather than guessing [6†L169-L174] [38†L139-L143] . (This goes beyond basic runtime verification: we’re gating actual state transitions or actions, not just monitoring a static property.)

Related frameworks: Other work (AgentSpec, workflow governors, etc.) ensures content safety or workflow integrity but **assumes each proposed action is self-contained** [38†L178-L186] . By contrast, HALO (and our proposal) explicitly tracks inter-component dependencies and refresh rules. The *AgentSpec* tool governs whether an action is safe given state, but does not handle *multi-part responses* or require fresh re-submission of stale parts [38†L198-L206] . Our approach is most similar to HALO’s component protocol [38†L228-L238] [15†L785-L793] , making that system the closest prior-art analogy.

Candidate Inventions and Claims

Based on the above, the most concrete invention is a **State Admission Engine** for provenance-ledgers or agent responses. In broad terms, one claim could be:

A computer-implemented method comprising:

- *reconstructing an observed candidate state from stored inputs;*
- *evaluating a set of typed admission predicates (e.g. byte-identity, authority-scope, temporal-scope, tracked-state-membership) against the candidate state;*
- *only after all predicates pass, promoting the candidate state to authoritative runtime state;*
- *otherwise inhibiting state promotion (fail-closed) and emitting a structured receipt listing which predicates failed and which consequences remain indeterminate.*

This narrows down from “abstract governance” to a **specific technical process** of typed predicates and fail-closed behavior (following USPTO guidance [6†L169-L174] [38†L139-L143]). It also matches HALO’s pattern (though HALO focuses on outputs, whereas our claim is about admitting *states or actions*).

A second claim family is **Consequence-Preserving Transition Validation**: for every gated event, compute exactly one lawful outcome. For example, for a transition T the system checks all admissible origins R with current observations, derives each candidate outcome $C(R)$, and ensures they coincide. If $|\{C(R)\}| = 1$, allow T ; else *withhold T as “NOT_IDENTIFIABLE.”* This is like taking HALO’s “largest supported set” but applied to state transitions.

A third family is **Counterfactual Independence Assay**: when admitting a consequence based on a basis B , automatically synthesize interventions (e.g. swap labels, remove prerequisites) to test that C truly follows from B and not circularly from the hypothesized outcome. This is an anti-circularity test not commonly found in existing systems. It combines ideas from causal identifiability (do-calculus) with runtime gating: a request’s “independence receipt” would contain the derivation tree, the interventions applied, and the recomputed consequences [14†L11-L16] [15†L785-L793] .

A fourth family is **Observation Selection for Identifiability**: at runtime, given a withheld consequence, compute the *optimal next observation* (query) that maximizes the expected reduction in consequence-

uncertainty. This is akin to active-learning (selecting the most informative measurement) but specifically using *remaining consequence heterogeneity* as the utility metric (e.g. information gain on C). The formula $q^* = \arg\max_q \frac{H_C(E) - \mathbb{E}[H_C(E|q)]}{\mathcal{C}(q)}$ ensures any new observation q is actually needed to resolve the gating. Such a $\frac{H_C(E) - \mathbb{E}[H_C(E|q)]}{\mathcal{C}(q)}$ minimum-missing-observation* rule is novel in the context of agent governance. (Notably, it can include “no-observation” as an option when cost outweighs benefit.)

Embodiment & Prototypes

We sketched minimal algorithms to validate these claims. For example, pseudocode for consequence gating:

```
# Given: current observation o, desired transition with outcome C
E = {R | observe(R) == o}      # compatible world-set
Gamma = {derive(R) for R in E} # independent consequence derivations
if |Gamma| == 1:
    EXECUTE(C) # invariant consequence
else:
    WITHHOLD("NOT_IDENTIFIABLE")
```

This already goes beyond a standard monitor: it gates execution (not just logs it). To capture predicate receipts, we would extend each predicate check to produce a struct

`{predicateID, evidence, status, reopeningRule}` rather than a boolean. These could be serialized so that replaying the same state with changed assumptions reproduces the same admission decision (preventing “receipt inflation”).

As a concrete test, we would simulate the existing failure mode. For instance, call AWS `CreateVolume` with various payloads and verify that only a correctly structured request is admitted. A unit test could try both AZ and AZId:

```
# Pseudocode for testing admission
state = candidate_state(include_both=False)
assert admission_engine.admit(state) == False # fail, missing fields
state = candidate_state(include_AZ=True)
assert admission_engine.admit(state) == True  # succeed, correct shape [40†L39-L43]
state = candidate_state(include_AZId=True)
assert admission_engine.admit(state) == True  # succeed, alternate shape
```

This demonstrates “replay-time re-admission”: the engine recomputes admissions if state or context changes, rather than assuming past outcomes are still valid.

We would also prototype the swap-test: given two candidate states R_a, R_b in the same observation class, swap their labels and check that the independently derived C values swap. Passing this swap-assay confirms independence [15†L785-L793]. If not, the admission record marks the basis as tainted.

Claims Sketches and Filing Recommendation

Claim 1 (Typed State Admission): A computer-implemented method as above (typed predicates gate candidate state to authority, fail-closed). Support: we see no exact prior art; it's a concrete data-structure/process claim (improves correctness and safety of automation). Potential prior art: HALO (admission of structured outputs) [38†L139-L143] [15†L785-L793], but HALO admits components not whole state snapshots. Also some security monitors use predicates, but none to *admit or reject state* based on multi-faceted checks.

Claim 2 (Consequence Identifiability Gate): A method that, given an observational equivalence class E and candidate consequence C , allows C only if $|\{C(R) \mid R \in E\}| = 1$. Support: this concretely ties execution control to a computational test of outcome invariance, not present in standard RV (which only reports inconclusive). Prior art: partial observability RV [6†L169-L174], but no system that stops a transition based on outcome divergence. We must ensure embodiment described (e.g. algorithm above) and highlight improvement (for safety, prevents accidental forks).

Claim 3 (Counterfactual Independence Receipt): A method of verifying a proposed basis B by applying intervention(s) (e.g. label-swap) to generate new states and re-deriving consequences, including packaging `{basis, interventions, new_outcomes}` as an audit. Support: drawing on HALO's idea of "localized obligations" [15†L785-L793] but making it explicit. This is novel; most systems either trust the agent or use static analysis.

Claim 4 (Observation Query Engine): A method that, given current uncertainty in C , computes an optimal next query q^* to ask the environment (based on expected information gain about C and a cost model). This is an innovative cross of active learning with AI governance. While experiment design is known in ML, tying it to gating enforced execution is new.

Claim 5 (Execution Receipt/Replay Preservation): An execution receipt scheme where each admitted action or state writes a signed record of evidence, including which predicates passed/failed. Upon replay (or re-query), the receipt proves the action was lawfully authorized without blindly replaying all checks. Prior art: the idea of execution records exists (e.g. prompting solutions [28†L141-L149]), but tying it to predicate-based admission is new.

We recommend pursuing **Claims 1–3 immediately** (highest value and concreteness), with strong supporting prototypes. Claims 4–5 are valuable but riskier (they relate to broader active learning / logging ideas). All claims should be written narrowly (e.g. specify "state admission engine", "typed predicates", "failure receipts"), to fit the latest USPTO guidance against abstract ideas [6†L169-L174] [38†L139-L143].

Priority for filing: 1) Typed State Admission Engine (highest critical path), 2) Consequence Identifiability Gate, 3) Counterfactual Independence Receipt. The AWS volume example provides concrete evidence of benefit: without admission, we accept multiple semantically different volumes as "complete". With the engine, only the correct structured request passes.

All candidate claims should be checked against existing governance/policy frameworks (e.g. Akeyless runtime authority, agent frameworks) to carve out the specific *combination* of admission, component

gating, and re-interrogation. The data above and our prototypes demonstrate **enabled embodiments** (e.g. HALO-like code, AWS API testing) that implement these concepts in practice.

Summary: Our analysis shows that the new **admission-centric architecture** (candidate state → typed predicates → authoritative state) and **consequence-gating logic** are concrete technical improvements to AI agent execution. They differ markedly from prior work that only monitors outputs or enforces coarse policies. We should proceed to draft precise claims around these mechanisms, illustrating them with the minimal code sketches and test cases described above. Finally, once a stable implementation is in place (with the observed provider fix and predicate receipts), one *real inference* (e.g. model-run with actual data) should complete through this pipeline to validate the end-to-end path and serve as a live proof of concept.

Sources: The above ideas build on recent research in runtime verification and AI agent governance [6†L169-L174] [38†L139-L143] [15†L785-L793] , and on AWS’s documented API requirements [40†L39-L43] . These citations support the technical assertions and show alignment with best practices in software safety.
